

POSIX Thread Scheduling and Affinity

PGSCE – Programação Avançada de Sistemas Embebidos

António Barros

POSIX Threads: Scheduling and Priorities

System Dependencies

- The contents and techniques presented in these slides are **Linux-specific**.
- They rely heavily on Linux-specific extensions to the POSIX threads (pthreads) standard.
- **Key Dependencies:**
 - ▶ Linux-specific scheduler policies (e.g., SCHED_FIFO).
 - ▶ GNU extensions for CPU affinity (e.g., pthread_setaffinity_np).
 - ▶ Linux kernel scheduling events, which are essential for the described tracing and analysis tools.

Code examples may not be portable to other POSIX-compliant systems (e.g., macOS or BSD) without modification.

Thread Scheduling: pthread_setschedparam()

The standard way to set the scheduling policy and parameters for a thread:

```
#include <pthread.h>

int pthread_setschedparam(pthread_t thread, int policy, const struct sched_param *param);
```

- **Parameters:**
 - ▶ **thread:** The thread ID to modify.
 - ▶ **policy:** Scheduling policy (e.g., SCHED_FIFO, SCHED_RR, SCHED_OTHER).
 - ▶ **param:** Pointer to a sched_param structure containing the priority.
- **Return Value:** 0 on success, or an error number if an error occurs.

Scheduling Parameters and Priority

To use `pthread_setschedparam()`, you must first configure the priority structure:

```
struct sched_param param;  
param.sched_priority = 3; // Priority value (range depends on policy)  
  
pthread_setschedparam(pthread_self(), SCHED_FIFO, &param);
```

- **Note:** For real-time policies like `SCHED_FIFO`, the process usually requires root privileges or appropriate `CAP_SYS_NICE` capabilities.
- **Range:** Use `sched_get_priority_min()` and `sched_get_priority_max()` to find valid priority values for your policy.
 - ▶ The higher the value, the more urgent the priority.

Querying Priority Ranges: `sched_get_priority`

Before setting a priority, you should query the valid range for the chosen policy:

- **Importance:** The valid priority range for real-time policies is system-dependent.
- **Compliance:** Always validate your priority values against these boundaries to avoid `EINVAL` errors when calling `pthread_setschedparam()`.
- **Scope:** These functions are essential for writing portable code that adapts to different Linux kernel configurations.

Querying Priority Ranges: sched_get_priority

```
#include <sched.h>
#include <stdio.h>

int main()
{
    int min_prio = sched_get_priority_min(SCHED_FIFO);
    int max_prio = sched_get_priority_max(SCHED_FIFO);

    printf("SCHED_FIFO range: [%d, %d]\n", min_prio, max_prio);

    return 0;
}
```

Querying Priority Ranges: sched_get_priority

Output on Linux

```
user@machine PRASIE $ ./print_prio  
SCHED_FIFO range: [1, 99]  
user@machine PRASIE $
```

Output on macOS

```
user@machine PRASIE % ./print_prio  
SCHED_FIFO range: [15, 47]  
user@machine PRASIE %
```

Dynamic Priority Adjustment: pthread_setschedprio()

A more direct way to modify the priority of an existing thread without changing its current scheduling policy:

```
#include <pthread.h>

int pthread_setschedprio(pthread_t thread, int prio);
```

- **Parameters:**
 - ▶ **thread:** The thread ID to modify.
 - ▶ **prio:** The new priority value to be assigned.
- **Return Value:** 0 on success, or an error number if an error occurs.

Comparison: `setschedparam` vs `setschedprio`

While both functions adjust priorities, they serve different purposes:

- **`pthread_setschedparam()`:**
 - ▶ Allows changing **both** the scheduling policy (e.g., from `SCHED_OTHER` to `SCHED_FIFO`) and the priority.
- **`pthread_setschedprio()`:**
 - ▶ Changes **only** the priority.
 - ▶ If the priority is changed, the thread may be moved to the end of the list for its priority level in the scheduler's queue.
- **Privileges:** Both functions require appropriate permissions to set real-time priorities (e.g., `CAP_SYS_NICE`).

Sidenote for Linux Users: Observing Execution Workflow

1. Trace Capture (CLI)

Capture events and terminate with Ctrl+C. Generates **trace.dat** in the current directory.

```
$ sudo trace-cmd record -e sched_switch -e sched_wakeup
```

2. Visualisation (GUI)

Open with KernelShark to analyse:

- **Pre-emption:** Higher-priority thread taking over.
- **Latency:** Runqueue delays (Wakeup vs. Execution).
- **System Noise:** Interruptions by kworker or swapper (idle).

```
$ kernelshark trace.dat
```

Periodic Threads

Periodic Thread Lifecycle

To maintain a strictly periodic schedule, the thread must calculate its next activation time relative to the absolute start time, rather than relying on relative sleep durations.

1. **Initialisation:** Set scheduling configuration.
2. **Setup:** Capture the starting timestamp using `clock_gettime()`.
3. **Loop:**
 - ▶ Execute task workload.
 - ▶ Increment activation timestamp by the period duration.
 - ▶ Suspend execution using `clock_nanosleep()` with `TIMER_ABSTIME`, to prevent drift by ignoring the time spent performing the task.

Implementation: Absolute Timing Pattern

```
struct timespec next, period;
clock_gettime(CLOCK_MONOTONIC, &next);           // Initial activation time
period.tv_sec = 0; period.tv_nsec = 500000000L;  // Period = 50 ms

while (active) {
    do_work();                                   // 1. Perform workload

    next.tv_sec += period.tv_sec;                // 2. Calculate next absolute activation
    next.tv_nsec += period.tv_nsec;
    if (next.tv_nsec >= 1000000000L) {           // If nanoseconds exceed one second
        next.tv_nsec -= 1000000000L;            // remove one second from nano and
        next.tv_sec++;                          // add it to seconds.
    }
    // 3. Sleep until the absolute deadline
    clock_nanosleep(CLOCK_MONOTONIC, TIMER_ABSTIME, &next, NULL);
}
```

Thread Affinity

Querying CPU Information in Linux

Before configuring affinity or scheduling, it is essential to understand the underlying hardware topology and available resources.

- **Command-Line Tools:**

- ▶ `lscpu`: Provides a comprehensive summary of the CPU architecture (cores, sockets, threads per core, and cache sizes).
- ▶ `nproc`: A simple command that returns the number of processing units available to the current process.
- ▶ `cat /proc/cpuinfo`: Displays the raw kernel interface for detailed per-processor information.

- **Programmatic Methods (C/POSIX):** Useful for dynamically adjusting the number of threads based on hardware.

Querying CPU Information in Linux

Getting the Number of Online Processors in C

```
#include <unistd.h>
#include <sys/sysinfo.h>

// Method 1: POSIX standard
long n_cores = sysconf(_SC_NPROCESSORS_ONLN);

// Method 2: GNU extension
int n_procs = get_nprocs();
```

- **Note:** Always distinguish between *physical cores* and *logical processors* (hardware threads/hyper-threading) when planning task distribution. Always base your calculations on physical cores.

Thread Affinity: pthread_setaffinity_np()

Function used to bind a thread to a specific set of CPU cores:

```
#define _GNU_SOURCE
#include <pthread.h>

int pthread_setaffinity_np(pthread_t thread, size_t cpusetsize, const cpu_set_t *cpuset);
```

- **Parameters:**
 - ▶ **thread:** The thread to be bound to the CPUs.
 - ▶ **cpusetsize:** Size of the cpu_set_t structure.
 - ▶ **cpuset:** Pointer to the mask of CPUs the thread is allowed to execute on.
- **Return Value:** 0 on success, or an error number if an error occurs.

Linux Thread Affinity: The CPU Mask

Affinity is controlled using a `cpu_set_t` structure, which acts as a bitmask representing the available CPU cores.

- **Initialization:** `CPU_ZERO(&mask)` clears the mask (no CPUs selected).
- **Configuration:** `CPU_SET(core_id, &mask)` adds a specific core to the set.
- **Exclusion:** `CPU_CLR(core_id, &mask)` removes a specific core from the set.

Configuring the CPU Set

Before calling `pthread_setaffinity_np()`, you must initialise the `cpu_set_t` mask:

```
cpu_set_t cpuset;  
CPU_ZERO(&cpuset);      // Clear all CPUs from the set  
  
CPU_SET(0, &cpuset);     // Pin the thread to cores 0 and 2  
CPU_SET(2, &cpuset);  
  
pthread_setaffinity_np(pthread_self(), sizeof(cpuset), &cpuset);
```

- **Note:** The `_np` suffix stands for "non-portable", indicating this is a Linux-specific extension to the POSIX threads API.
- **Performance:** Affinity helps improve cache locality and reduces the overhead of thread migration between cores.

Performance Implications of Affinity

Setting thread affinity is a powerful tool for low-latency and high-performance applications:

- **Cache Locality:** By keeping a thread on the same core, the CPU's L1/L2 caches remain "warm", reducing expensive main memory accesses.
- **Reduced Context Switching:** Prevents the Linux scheduler from migrating threads across different cores, which involves costly cache invalidations.
- **Determinism:** Isolating real-time tasks to specific cores ensures that high-priority threads are not disrupted by non-critical background processes.

Over-constraining threads (e.g., pinning all threads to a single core) can lead to resource contention and decreased throughput. Always profile your application.